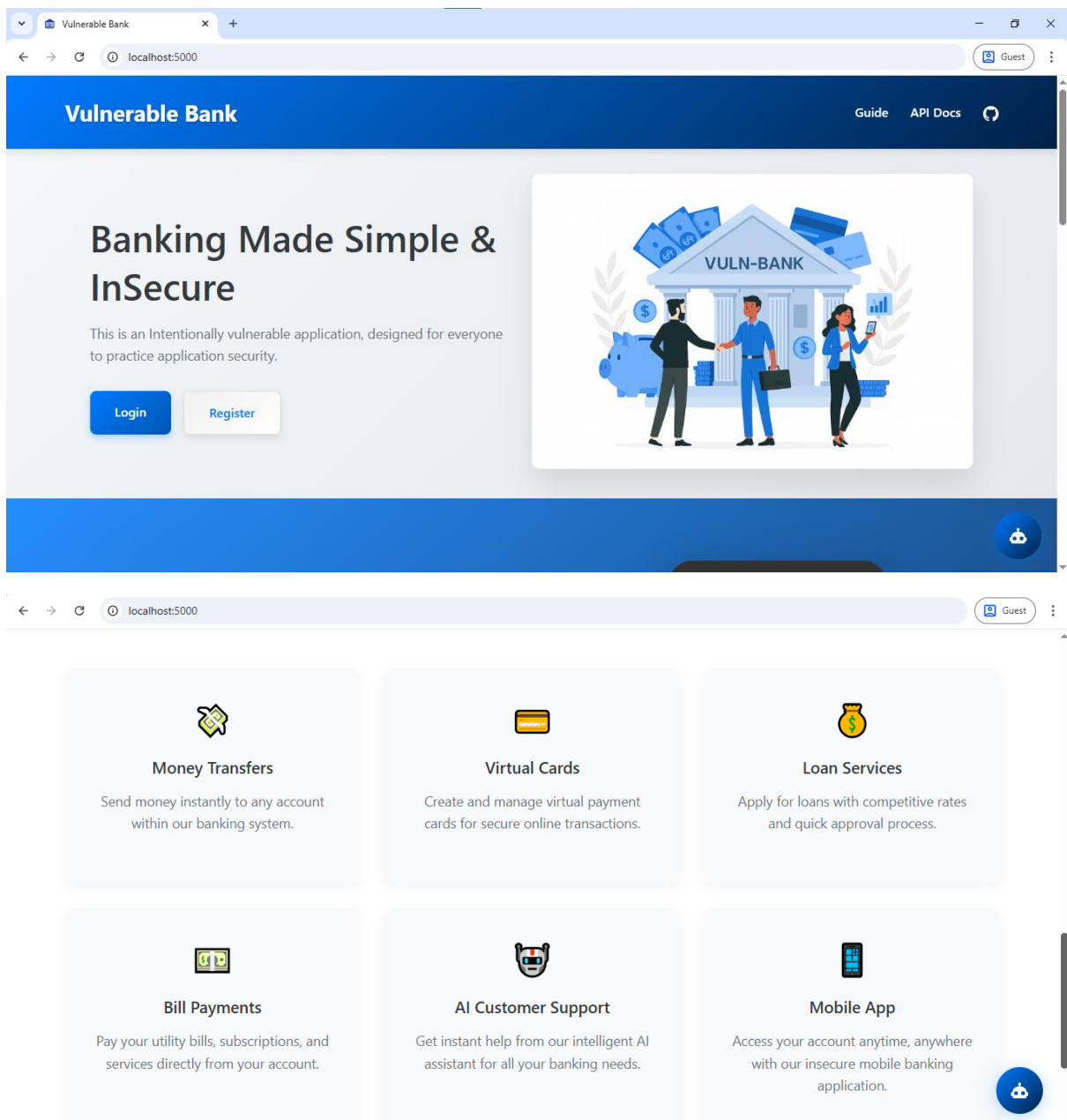



```
=> => naming to docker.io/library/vuln-bank-web:latest 0.0s
=> => unpacking to docker.io/library/vuln-bank-web:latest 2.1s
=> resolving provenance for metadata file 0.1s
[+] Running 5/5
  ✔ vuln-bank-web Built 0.0s
  ✔ Network vuln-bank_vuln_network Created 0.2s
  ✔ Volume vuln-bank_postgres_data Created 0.0s
  ✔ Container vuln-bank-db-1 Started 2.8s
  ✔ Container vuln-bank-web-1 Started 3.0s

SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        NAMES          STATUS          PORTS
425d7f6fac4e   vuln-bank-web  "python app.py"         About a minute ago   vuln-bank-web-1 Up About a minute  0.0.0.0:5000->5000/tcp,
0d64a470908f   postgres:13    "docker-entrypoint.s..." About a minute ago   vuln-bank-db-1  Up About a minute  0.0.0.0:5432->5432/tcp,
[::]:5432->5432/tcp

SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$
```



Inspect web container logs

```
docker logs vuln-bank-web-1 --tail 200
```

```
SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$ docker logs vuln-bank-web-1 --tail 200
Failed to connect to database (attempt 1/5): connection to server at "db" (172.18.0.2), port 5432 failed: Connection refused
Is the server running on that host and accepting TCP/IP connections?

Retrying in 2 seconds...
Failed to connect to database (attempt 2/5): connection to server at "db" (172.18.0.2), port 5432 failed: Connection refused
Is the server running on that host and accepting TCP/IP connections?

Retrying in 2 seconds...
Database connection pool created successfully
Database initialized successfully
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on all addresses.
  WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://172.18.0.3:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 741-727-406
172.18.0.1 - - [29/Oct/2025 02:26:49] "GET / HTTP/1.1" 200 -
172.18.0.1 - - [29/Oct/2025 02:26:49] "GET /static/style.css HTTP/1.1" 200 -
```

Shows debug mode enabled, debugger PIN exposed, DB connection attempts and later success, and 500 responses for login attempts.

SQLi-style test against login

```
SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$ curl -i -X POST "http://localhost:5000/login" \
> -H "Content-Type: application/x-www-form-urlencoded" \
> --data-urlencode "username=admin' OR '1'='1' -- " \
> --data-urlencode "password=anything" \
> -v
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host localhost:5000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:5000...
* Connected to localhost (::1) port 5000
* using HTTP/1.x
> POST /login HTTP/1.1
> Host: localhost:5000
> User-Agent: curl/8.14.1
> Accept: */*
> Content-Type: application/x-www-form-urlencoded
> Content-Length: 60
>
* upload completely sent off: 60 bytes
* HTTP 1.0, assume close after body
< HTTP/1.0 500 INTERNAL SERVER ERROR
HTTP/1.0 500 INTERNAL SERVER ERROR
< Content-Type: application/json
Content-Type: application/json
< Content-Length: 110
Content-Length: 110
< Content-Type: application/json
Content-Type: application/json
< Content-Length: 110
Content-Length: 110
< Access-Control-Allow-Origin: *
Access-Control-Allow-Origin: *
< Server: Werkzeug/2.0.1 Python/3.9.24
Server: Werkzeug/2.0.1 Python/3.9.24
< Date: Wed, 29 Oct 2025 03:06:46 GMT
Date: Wed, 29 Oct 2025 03:06:46 GMT
<
{
  "error": "'NoneType' object has no attribute 'get'",
  "message": "Login failed",
  "status": "error"
}
* shutting down connection #0

SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$ _
```

The crafted input triggered a server-side exception and a 500 response with an internal Python error message — evidence that unvalidated input reaches server logic (consistent with SQLi or other input-handling bugs).

Opened a shell into the web container and inspected files

```
SHARIKACOMPUTERS@DESKTOP-CQ0CGF2 MINGW64 ~/vuln-bank (main)
$ docker exec -it vuln-bank-web-1 bash
root@425d7f6fac4e:/app# ls
Dockerfile  README.md    ai_agent_deepseek.py  auth.py    docker-compose.yml  static
LICENSE.md  __pycache__  app.py                database.py requirements.txt     templates
root@425d7f6fac4e:/app#
```

```
# Create a connection pool
connection_pool = None

def init_connection_pool(min_connections=1, max_connections=10, max_retries=5, retry_delay=2):
    """
    Initialize the database connection pool with retry mechanism
    Vulnerability: No connection encryption enforced
    """
    global connection_pool
    retry_count = 0

    while retry_count < max_retries:
        try:
            connection_pool = psycpg2.pool.SimpleConnectionPool(
                min_connections,
                max_connections,
                **DB_CONFIG
            )
            print("Database connection pool created successfully")
            return
        except Exception as e:
            retry_count += 1
            print(f"Failed to connect to database (attempt {retry_count}/{max_retries}): {e}")
            if retry_count < max_retries:
                print(f"Retrying in {retry_delay} seconds...")
                time.sleep(retry_delay)
            else:
                print("Max retries reached. Could not establish database connection.")
                raise e
```

```
root@425d7f6fac4e:/app# sed -n '1,200p' database.py
import os
import psycpg2
from psycpg2 import pool
from datetime import datetime
import time

# Vulnerable database configuration
# CWE-259: Use of Hard-coded Password
# CWE-798: Use of Hard-coded Credentials
DB_CONFIG = {
    'dbname': os.getenv('DB_NAME', 'vulnerable_bank'),
    'user': os.getenv('DB_USER', 'postgres'),
    'password': os.getenv('DB_PASSWORD', 'postgres'), # Hardcoded password in default value
    'host': os.getenv('DB_HOST', 'localhost'),
    'port': os.getenv('DB_PORT', '5432')
}
```

```

def get_connection():
    if connection_pool:
        return connection_pool.getconn()
    raise Exception("Connection pool not initialized")

def return_connection(connection):
    if connection_pool:
        connection_pool.putconn(connection)

def init_db():
    """
    Initialize database tables
    Multiple vulnerabilities present for learning purposes
    """
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            # Create users table
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS users (
                    id SERIAL PRIMARY KEY,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL, -- Vulnerability: Passwords stored in plaintext
                    account_number TEXT NOT NULL UNIQUE,
                    balance DECIMAL(15, 2) DEFAULT 1000.0,
                    is_admin BOOLEAN DEFAULT FALSE,
                    profile_picture TEXT,
                    reset_pin TEXT -- Vulnerability: Reset PINs stored in plaintext
                )
            ''')

```

```

            # Create loans table
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS loans (
                    id SERIAL PRIMARY KEY,
                    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
                    amount DECIMAL(15, 2),
                    status TEXT DEFAULT 'pending'
                )
            ''')

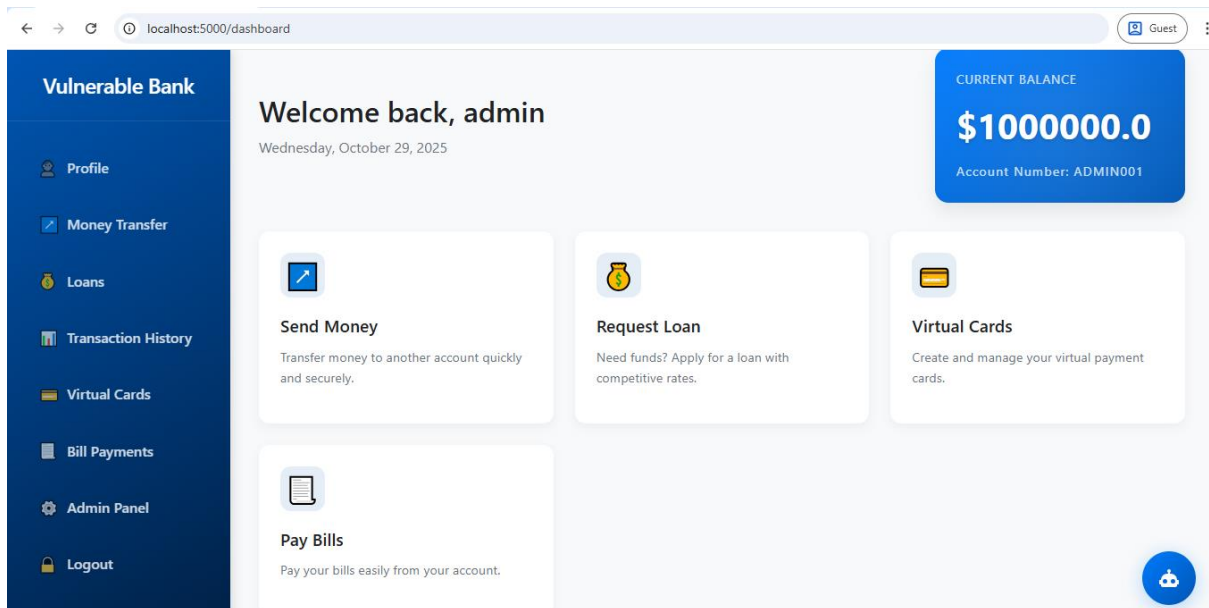
            # Create transactions table
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS transactions (
                    id SERIAL PRIMARY KEY,
                    from_account TEXT NOT NULL,
                    to_account TEXT NOT NULL,
                    amount DECIMAL(15, 2) NOT NULL,
                    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    transaction_type TEXT NOT NULL,
                    description TEXT
                )
            ''')

            # Create virtual cards table
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS virtual_cards (
                    id SERIAL PRIMARY KEY,
                    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
                    card_number TEXT NOT NULL UNIQUE, -- Vulnerability: Card numbers stored in plaintext
                    cvv TEXT NOT NULL, -- Vulnerability: CVV stored in plaintext

```

Source-level evidence of:

- Hard-coded default DB credentials in DB_CONFIG default values.
- Table schema that stores password and reset_pin as plain text.
- Default admin account inserted with plaintext admin123.



Findings

1) Hard-coded administrative credentials — High

- **Evidence:** `database.py` contains `INSERT ... ('admin', 'admin123', 'ADMIN001', ...)`.
- **Impact:** Anyone who knows these defaults can log in as admin and perform privileged actions. Immediate takeover risk.

2) Plaintext credential storage & plaintext reset PINs — High

- **Evidence:** `database.py` shows `password TEXT NOT NULL` and `reset_pin TEXT`. The insertion of `'admin123'` in plaintext confirms password storage in cleartext.
- **Impact:** Database compromise yields readable user credentials and reset codes; enables account takeover and credential reuse attacks.

3) Debug mode & information disclosure — Medium → High

- **Evidence:** `docker logs ...` shows `Debug mode: on`, `Debugger PIN: 741-727-406`, and `Werkzeug/Python` server version details. The login test returned a JSON with internal Python error (`'NoneType' object has no attribute 'get'`).
- **Impact:** Exposes internal implementation details, aiding automated exploitation and reconnaissance; if debugger is reachable, it can enable remote code execution.

4) Input-handling flaw consistent with SQL injection risk — High (likely)

- **Evidence:** SQLi-styled payload against `/login` caused a server-side exception (500) and internal error message. That indicates user-supplied input is making the server fail — consistent with unsanitized SQL usage.
- **Impact:** If SQL injection is exploitable, it could allow authentication bypass, data exfiltration, or DB manipulation.